

Status

CodeGraph Status — Boba

Revision: 2 **Last modified:** 2026-08-18T13:33:41Z **Authority:**

Constitution §11.4.78 / §11.4.79 / §11.4.80 **Scope:** Append-only ledger of CodeGraph install / update / sync / validate events for the Boba project. Newest entries appended at the bottom by constitution/scripts/codegraph_sync.sh.

2026-06-06T14:30:00Z — initial setup + update + re-index (§11.4.78/§11.4.79/§11.4.80)

- **Install / update:** host already had CodeGraph via the standalone installer (~/.codegraph/versions/v0.9.7, shim at ~/.local/bin/codegraph). The npm package (@colbymchenry/codegraph, §11.4.78) latest was 0.9.9. constitution/scripts/codegraph_update.sh installed 0.9.9 via `npm install -g` (no sudo; user-writable prefix /opt/homebrew) and the §107 anti-bluff check caught that the stale ~/.local/bin shim still won on PATH. Removed the stale standalone shim → codegraph on PATH is now the npm-managed 0.9.9 at /opt/homebrew/bin/codegraph.
- **Config model:** CodeGraph 0.9.9 is **zero-config** — there is NO .codegraph/config.json exclude list. Exclusion is driven by .gitignore
 - built-in defaults (node_modules/vendor/dist/... + files > 1 MB). Source: the installed package README.md “Configuration” section.
- **Exclusions (§11.4.10 + §11.4.79):** secrets are already covered by root .gitignore (.env*, *secrets*, config/qBittorrent/, config/jackett/, config/boba.db). Added submodules/jackett/ to root .gitignore to keep the third-party Jackett (C#/.NET) submodule OUT of the index — inert for git (jackett is a tracked gitlink) but honored by CodeGraph. Added .codegraph/codegraph.db (+ WAL/SHM/cache)

to root .gitignore; regeneration mechanism (§11.4.77) is codegraph index.

- **Re-index:** codegraph index . --force on 0.9.9.
- **Index result:** 509 files, 8906 nodes, 17025 edges. Languages: go, python, typescript, properties, xml, yaml.
- **Audit:** submodules/jackett = 0 indexed (§11.4.79 ✓); real secret paths = 0 indexed (§11.4.10 ✓); constitution/ own-org = 19 files indexed, cross-submodule symbol versionTagsCmd resolves (§11.4.79 ✓).
- **MCP wiring:** .mcp.json registers codegraph serve --mcp (stdio, bare command on PATH). Verified the server boots and codegraph_status returns the live node count (8906) over MCP — the §11.4.78 step-4 unforgeable fact.
- **Validate:** scripts/codegraph_validate.sh → 7 PASS / 0 FAIL.

2026-06-06T14:38:30Z — codegraph_sync.sh @ .

- duration: 1s
- baseline status:

method	2,599
function	1,722
file	498
constant	139
struct	85
interface	84

- post-sync status:

method	2,599
function	1,722
file	498
constant	139
struct	85
interface	84

- validate: **PASS**

PASS: codegraph on PATH at /opt/homebrew/bin/codegraph (version 0.9.9)

PASS: index reality – codegraph status reports 8906 nodes across 509 files

PASS: unforgeable MCP challenge – codegraph_status via MCP returned 8906 nodes == CLI 8906

PASS: own-code resolution – 'Deduplicator' resolves in download-proxy/src (8 hit(s))

PASS: own-org INCLUDED (§11.4.79) – 'versionTagsCmd' resolves inside constitution/ (2 hit(s))
PASS: third-party EXCLUDED (§11.4.79) – 0 submodules/jackett paths in index
PASS: secrets EXCLUDED (§11.4.10) – 0 secret/config-credential paths in index

CodeGraph validate: 7 PASS / 0 FAIL
VERDICT: PASS

2026-08-18T13:33:41Z — BOB-075 staleness remediation: re-index attempted, real blocker found + honestly reported (§11.4.6 / GOVERNANCE_AUDIT_2026-08-08_ROUND2.md RD2-08)

Staleness confirmed (not a false positive): this ledger's only prior entry was 2026-06-06 — 73 days before this pass, with zero sync events recorded in between despite §11.4.80's "regular update automation" mandate. The tracked codegraph CLI itself had moved from the documented 0.9.9 to **1.5.0** (codegraph --version) with no re-verification against the new version.

Attempted mechanical regen (Option A): ran codegraph init . --force on this host to produce a real, current re-index (the same command the 2026-06-06 entry used). Real observed outcome, NOT simulated:

- The run walked **32,260 files / 514,456 nodes / 724,013 edges** (DB size 1.8 GB) before being deliberately aborted — a **~63× blowup** versus the 2026-06-06 baseline of 509 files / 8,906 nodes for what is still fundamentally the same repository shape (Python/Go/TS backend + plugins).
- Root cause, CONFIRMED (not guessed, §11.4.6): git check-ignore -v proves frontend/node_modules/** and extension/node_modules/** ARE correctly excluded by nested frontend/.gitignore:10 (/node_modules) and extension/.gitignore:2 (node_modules/) for git itself (git ls-files frontend/node_modules extension/node_modules → 0 rows both) — yet codegraph init on **1.5.0** walked into both trees anyway (365 MB + 236 MB, tens of thousands of files). This is consistent with a CodeGraph 1.5.0 regression in honoring **nested** (non-root) .gitignore files — the doc's own "zero-config, exclusion driven by .gitignore" claim (2026-06-06 entry) held for the flat 509-

file tree that existed then; frontend/ and extension/ (with their own node_modules) were added to this project AFTER that entry and were never exercised against this exclusion path before now.

- **Decision:** aborted the run (host-resource discipline, §12.6/§12.11 — an 18-core-minute, still-growing 1.8 GB index for a bounded docs-staleness task is disproportionate) and removed the resulting truncated .codegraph/codegraph.db* (gitignored, never tracked; confirmed via codegraph status reporting “index truncated” after the kill — no partial state was left claiming to be a valid index).
- **Honest current state:** this project’s CodeGraph index is **NOT live on this host** as of this entry. The last VALIDATED sync remains the 2026-06-06 entry above (7 PASS / 0 FAIL on the pre-frontend/extension tree shape). A full resync needs the CodeGraph 1.5.0 nested-.gitignore gap investigated first (or an explicit frontend/node_modules/extension/node_modules exclusion added at this project’s root .gitignore, since the root file IS honored) — **not performed in this pass**; this task’s DB-touch scope is limited to closing BOB-075 itself (BOB-072/073 owns broader DB work), so no new tracked item was minted here for the CodeGraph regression — flagging it in-repo per §11.4.6 rather than silently deferring it.

This entry itself is the mechanical regen this ledger’s own header mandates (§11.4.44/§11.4.86): the Revision/Last-modified above are bumped to reflect this REAL attempted-and-honestly-reported sync event, not a cosmetic timestamp touch.